Jay Wanjare
wanjarejay25@gmail.com

Detailed Technical Approach

Objective

The goal of this project was to develop a recommendation system for assessments using data scraping, semantic embeddings, and similarity-based retrieval methods. The system takes various attributes such as test titles, descriptions, and categories and uses them to recommend relevant assessments to users.

Data Collection and Scraping

Scraping Process:

1. **Use requests to fetch the HTML content of the web pages.**
2. **Parse the HTML with BeautifulSoup to extract data from specific HTML tags.**
3. **Store the extracted data in a structured format, such as CSV or database tables.**

Data Preprocessing and Encoding

- **Text Combination: Merge the fields title, description, and other relevant metadata into one column called combined_description.**
- **One-Hot Encoding: Categorical data, such as test type and remote availability, was converted into binary vectors to represent them numerically.**

Embedding Generation with Google Gemini

After preprocessing, the system uses Google Gemini to generate embeddings for each assessment. These embeddings represent the semantic meaning of each assessment, which is essential for making accurate recommendations. The combined_description was passed through Gemini, generating high-dimensional vectors that capture the underlying content relationships between assessments.

Each assessment's embedding was stored in a database, allowing for efficient similarity comparisons when new queries arrive. When a user submits a query, its combined_description is passed through Gemini, and a corresponding embedding is generated to compare with the stored embeddings.

Jay Wanjare
wanjarejay25@gmail.com

Recommendation Engine

1. **Query Processing: When a user submits a query, the system combines the text fields (title, description, etc.) and processes them using Google Gemini to generate an embedding.**
2. **Similarity Matching: The query embedding is then compared against all stored embeddings using Cosine Similarity. The most similar assessments are identified and returned as recommendations.**
3. **Efficiency: By precomputing embeddings for all assessments, the system ensures that real-time queries are handled efficiently, without needing to reprocess the embeddings each time.**

Online Deployment

The recommendation engine was designed for online deployment to handle real-time queries. FastAPI was used to expose the system as an API, making it accessible for integration into other applications or websites.

- **Scalability: New assessments can be added to the system by scraping additional data, generating embeddings for the new assessments, and updating the database.**
- **Real-Time Adaptability: The system dynamically handles user queries, providing real-time responses based on the stored embeddings.**
